

Arithmétique des ordinateurs

Jorel Raphaël

Pour commencer

Dans cette petite étude, nous allons nous intéresser à 12 algorithmes de sommation d'entiers. Ces 12 algorithmes sont évidemment différents, le but étant d'observer la perte de précision causée par les additions successives. Pour cela, nous disposons de 44 séries de données différentes stockées dans des fichiers, chacun contenant 100 nombres flottants, le conditionnement de la somme de ces derniers et le résultat exact, afin de mesurer la perte de précision de chacun des algorithmes.

Je rappelle juste les 12 algorithmes de sommation, afin de pouvoir suivre les explications suivantes.

- A1. dans l'ordre croissant des indices : $((x_0 + x_1) + x_2) + \dots$,
- A2. dans l'ordre décroissant des indices,
- A3. opérandes positifs puis négatifs dans l'ordre croissant des indices,
- A4. opérandes négatifs puis positifs, dans l'ordre croissant des indices,
- A5. somme partielle S_+ des opérandes positifs, puis somme partielle S_- des opérandes négatifs chacune dans l'ordre croissant des indices, puis somme de S_+ et S_- ,
- A6. ordre croissant des valeurs des opérandes,
- A7. ordre décroissant des valeurs des opérandes,
- A8. ordre croissant des valeurs absolues des opérandes,
- A9. ordre décroissant des valeurs absolues des opérandes,
- A10. ordre croissant des valeurs des opérandes positifs, puis ordre croissant des valeurs des opérandes négatifs,
- A11. addition récursive par paire (pairwise) : $((x_0 + x_1) + (x_2 + x_3)) + ((x_4 + x_5) + (x_6 + x_7))$ pour $n = 8$ par exemple,
- A12. supprimer les deux opérandes de plus petite valeur absolue, les additionner, ajouter cette somme comme un nouvel opérande et recommencer (addition des deux plus petites valeurs absolues, ...) jusqu'à ce qu'il n'y ait plus d'opérande à additionner.

Pour rappel, tout au long de cette étude s représente le calcul exact et $\hat{s}$ représente le calcul d'un algorithme.

1. Les algorithmes

Vérifions tout d'abord que ces algorithmes diffèrent seulement par l'ordre de leur sommes partielles.

Ordres de sommation

Ces 12 algorithmes décrivent des ordres de sommation différents. En effet, les algorithmes **A1**, **A2**, **A3**, **A4**, **A5** et **A11**, n'impose pas d'ordres numériques sur les données, tandis que les algorithmes **A6**, **A7**, **A8**, **A9** et **A10** et **A12** en impose un. Les ordres de sommation de ces deux "classes" d'algorithmes sont forcément différents car les données à sommer ne sont pas ordonnées.

En ce qui concerne les premiers cités, **A1** et **A2** additionnent les valeurs dans des sens opposés. **A3** commence par les positifs, puis les négatifs, **A4** fait le contraire, il paraît évident que l'ordre de sommation de ces 4 algorithmes sont tous différents. **A5**, lui, fait des sommes partielles, contrairement aux deux précédents qui somme à la suite les positifs et négatifs, ou inversement. **A11** additionnant par paire, son parenthésage donné dans sa description nous indique clairement son ordre de sommation, qui est complètement différent des 5 précédents.

A6 et **A7** somme, respectivement, dans les ordres croissant et décroissant, nul besoin de prouver qu'ils parenthésent différemment le calcul de $\hat{s}$. **A8** et **A9** font de même, mais sur les valeurs absolues. S'il n'y avait pas de valeurs négatives, ils reviendraient à faire la même chose que les précédents, mais ce n'est pas le cas. De ce fait, les nombres négatifs et positifs peuvent se retrouver "mélangés", tout en étant trié dans l'ordre croissant de leur valeur absolue. Les ordres de sommation sont donc bien différents. **A10** est un peu différent, mais il est le seul des algorithmes qui commence à sommer par la plus petite valeur positive. Ceci nous permet d'affirmer que son ordre de sommation n'est pas le

même que les autres. Pour finir, **A12** est un peu particulier, puisqu’il remet chaque somme partielle dans les opérandes. Comme aucun des algorithmes ne fait cela à part lui, il est impossible qu’un autre ait le même ordre de sommation que lui.

Vérification

Pour vérifier que ces algorithmes sont différents de par leurs sommes partielles, il suffirait de conserver l’ordre des opérandes de départ, et de retenir dans quel ordre ces derniers sont ajoutés pour chaque algorithme. Ensuite, il faudrait comparer ces ordres.

Inverse-stable

Ces algorithmes sont tous inverses-stables car ils se basent sur l’addition telle que définie dans la norme IEEE-754, qui est inverse-stable.

2. Erreurs

Maintenant, regardons les erreurs générées par les additions successives de chaque algorithme. Nous allons nous intéresser au nombre de bits significatifs entre s et $\hat{s}$, c’est-à-dire le nombre de bits de poids fort communs à la mantisse de s et de $\hat{s}$. Le calcul de l’erreur relative en bits se fait grâce au calcul

$$ErrRelBits(s, \hat{s}) = \log_2(ErrRel(s, \hat{s})) = \log_2\left(\frac{|s - \hat{s}|}{|s|}\right)$$

Pour vérifier cela de manière algorithmique, nous pouvons penser à faire un petit programme qui compare les bits de la mantisse de s et celle de $\hat{s}$ en partant du bit de valeur la plus importante (2^{-1}) et en allant vers le bit de valeur la moins importante (2^{-53} en **b64**). Dès qu’un bit diffère, l’algorithme se termine et renvoie la position à laquelle il s’est arrêté.

Le nombre de bits perdus est tout simplement le nombre de bits total de la mantisse auquel on retranche le nombre de bits communs entre s et $\hat{s}$. En **b64**, cela nous donne

$$NbBitsPerdus(s, \hat{s}) = LongueurMantisse(b64) - ErrRelBits(s, \hat{s}) = 53 - ErrRelBits(s, \hat{s})$$

3. Comparaisons

Comparons maintenant la perte de précision de tous ces algorithmes. Voici deux graphiques fait à partir des 44 fichiers de données, basés sur 11 conditionnements de somme différents. Le premier graphique représente les moyennes des nombres de bits perdus par conditionnements et par algorithmes, le second montre les nombres de bits perdus maximaux.

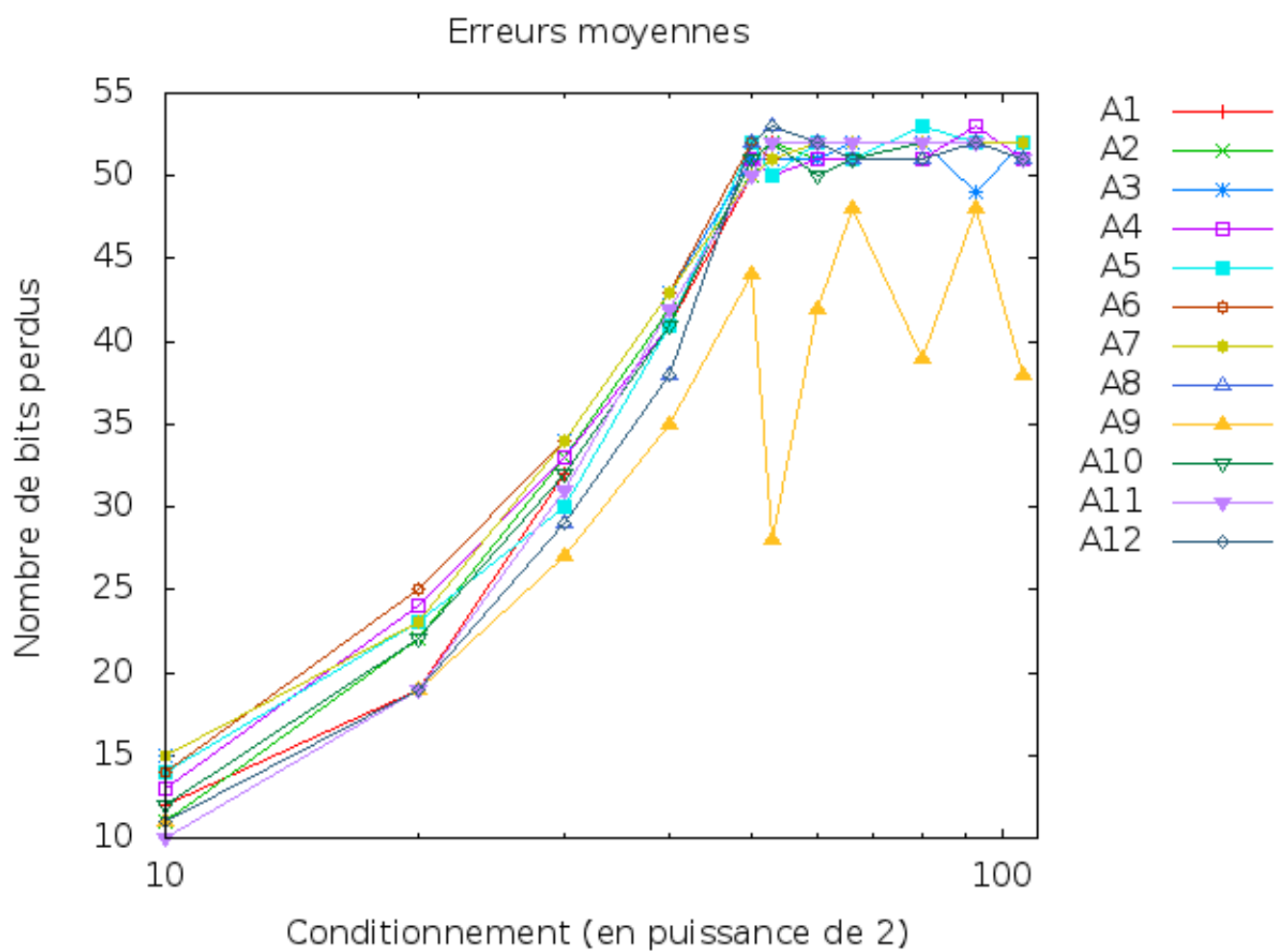


FIGURE 1 – Nombres de bits perdus en moyenne

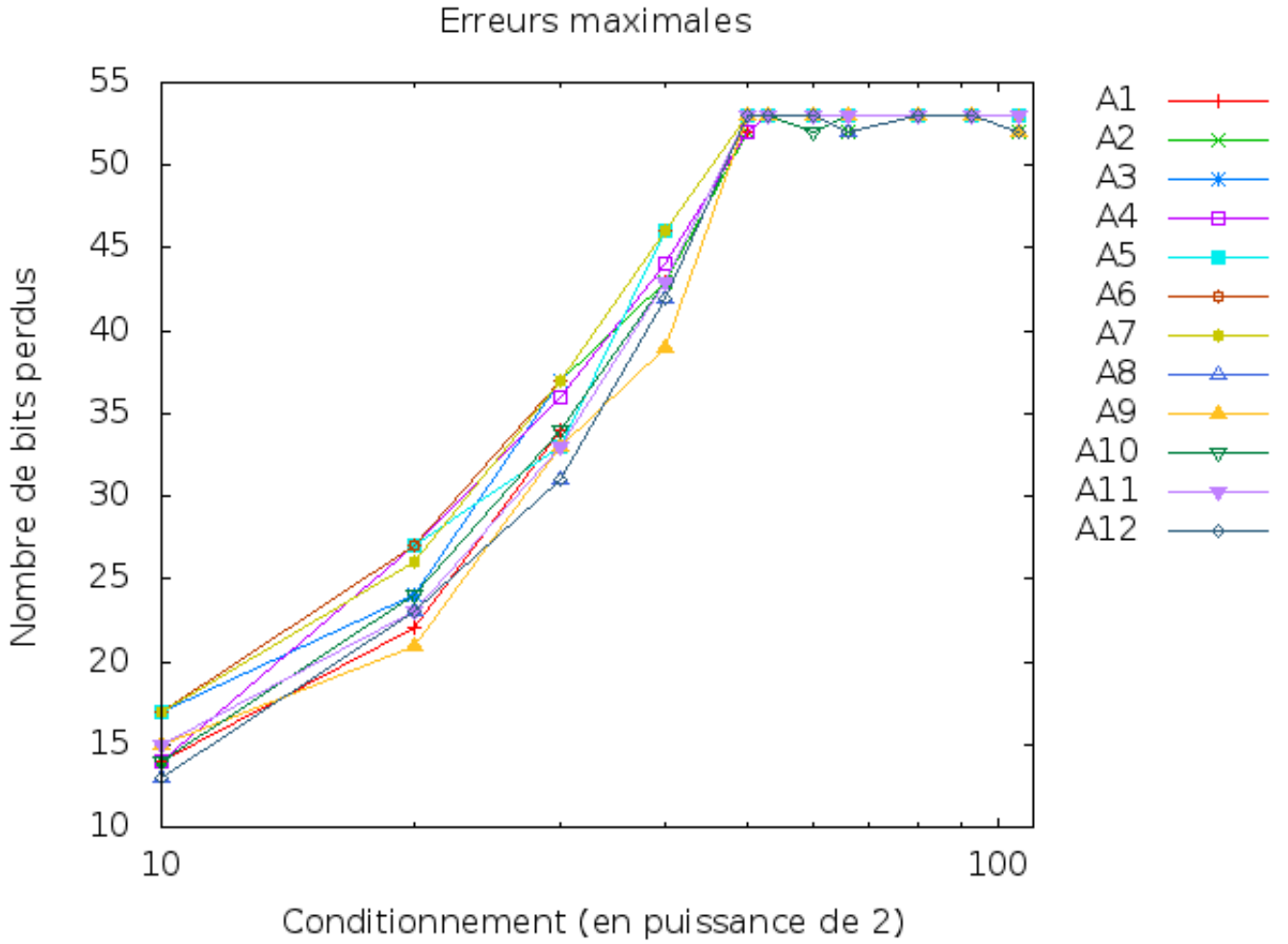


FIGURE 2 – Nombres de bits perdus maximaux

Nous remarquons que toutes les courbes suivent (à peu près) les mêmes variations, que ce soit pour les nombres de bits perdus en moyenne, comme ceux maximaux. Seul l'algorithme A9 sort du lot dans le premier graphique. Finalement, nous voyons que quelque soit la manière de sommer, nous aurons toujours plus ou moins les mêmes erreurs en moyenne, bien que certaines manières soient un peu plus précises que d'autres. Il est intéressant de noter que l'algorithme A1 n'est pas celui qui est le moins précis, alors qu'il est le plus simple à réaliser et le plus intuitif. Nous pouvons aussi voir que, certaines fois, imposer un ordre numérique sur les données ne rend pas le calcul plus précis (algorithme A6), d'autres fois si (algorithme A9), et que le conditionnement des données peut faire varier étrangement les erreurs de chaque algorithmes. L'algorithme A8, par exemple, est plus précis que la plupart des autres jusqu'à 2^{50} , devient le moins précis autour de 2^{50} , puis revient dans la moyenne générale des autres algorithmes. Cependant, l'algorithme A9 paraît être le plus intéressant en moyenne.

4. Majoration

La majoration de l'erreur relative d'un algorithme inverse-stable se fait grâce à la formule

$$ErrRel(x, \hat{x}) \leq \text{conditionnement} \times u$$

u : précision d'arrondi ($= \epsilon_{machine} = 2^{-53}$ en **b64**).

Comme nous effectuons $n - 1$ additions (pour n nombres à sommer), la majoration de l'erreur

relative de ces algorithmes est

$$ErrRel(s, \hat{s}) \leq (n - 1) \times \textit{conditionnement} \times u$$

Malheureusement, et pour une raison qui m'échappe, je n'ai pas réussi à faire figurer cette majoration sur les graphiques, car les résultats de la formule précédente appliquée aux données de chaque fichier sont complètement faux. En effet, je trouve la plupart du temps des majorations négatives. Donc j'imagine que j'ai mal appliqué la formule, ou mal l'adapté dans le programme. Dommage...